XX 大学

毕业设计

题　　目：基于 Spring Boot 的体育用品销售网站的设计与实现

学　　部：信息工程学院

专　　业：计算机科学与技术

姓　　名：某　某

学　　号：2022XXXXXXXX

指导教师：某　老师

职　　称：教　授

日　　期：2026 年 5 月

摘　要

随着全民健身热潮的全面兴起，运动爱好者对体育用品的线上采购需求持续呈爆发式增长。体育用品网店长期面临运营成本高、夜间客服缺位等挑战。设计并实现了一套基于 Spring Boot 与 Vue3 的体育用品销售网站，以买家与管理员两类用户为核心架构。普通买家可完成账号注册登录、器材搜索、购物管理、下单支付以及售后申请等操作；管理员则可通过后台系统，执行商品上下架管控、库存动态维护、订单处理、退换货精准审核及平台公告轮播图实时发布等工作。

该平台在前台用户交互层面接入了 Deep Seek 大语言模型作智能客服模块。可实时抓取用户历史订单数据提供查询服务，填补非工作时间的服务真空。为中小型体育用品商户提供了一套智能化解决方案。

关键词：体育用品销售；Spring Boot；Vue3；Deep Seek

ABSTRACT

With the comprehensive rise of the nationwide fitness trend, the online purchasing demand for sports goods among enthusiasts has continued to grow explosively. Sports goods online stores have long faced challenges such as high operating costs and the absence of customer service during nighttime hours. A sports goods sales website based on Spring Boot and Vue3 was designed and implemented, centered on two types of users: buyers and administrators. Regular buyers can complete operations such as account registration and login, equipment search, shopping management, order placement and payment, and after-sales service requests; administrators, on the other hand, can use the backend system to manage product listings, dynamically maintain inventory, process orders, accurately review returns and exchanges, and publish platform announcements and carousel images in real time.

At the front-end user interaction level, the platform integrates the Deep Seek large language model as an intelligent customer service module. It can retrieve users' historical order data in real time to provide query services, filling the service gap during non-working hours. This provides a smart solution for small and medium-sized sports goods merchants..

Key words: Sports Goods Sales; Spring Boot; Vue3; Deep Seek

目　录

1 绪论 1

1.1 研究背景 1

1.2 现有系统对比分析 1

1.3 开发的目的和意义 1

1.4 开发内容 2

2 关键技术介绍 3

2.1 后端核心技术 3

2.1.1 Spring Boot 特性与优势 3

2.1.2 MyBatis-Plus 数据持久化 3

2.1.3 MySQL 数据库技术 3

2.2 前端核心技术 4

2.2.1Vue3 组合式 API 4

2.2.2Vite 构建与组件化开发 4

2.2.3Axios 前后端交互 4

2.3 安全与规范标准遵循 4

3 系统分析 5

3.1 需求分析 5

3.2 技术可行性 5

3.3 运行可行性 5

3.4 经济可行性 6

3.5 操作可行性 6

4 系统设计 7

4.1 体育用品销售网站功能结构设计 7

4.2 体育用品销售网站数据库设计 7

4.2.1 数据库设计 7

4.2.2 数据库表设计 10

5 系统实现 14

5.1 管理员模块 14
5.1.1 管理员登录的实现 14
5.1.2 商品管理的实现 15
5.1.3 用户管理的实现 16
5.1.4 订单管理的实现 17
5.1.5 管理员受理退换货实现 18
5.1.6 轮播图管理实现 19
5.1.7 公告管理实现 20
5.2 前台用户模块 21
5.2.1 用户注册和登录实现 22
5.2.2 商品浏览和搜索功能实现 23
5.2.3 购物车功能实现 25
5.2.4 订单下单实现 26
5.2.5 支付功能实现 27
5.2.6 智能客服实现 28
5.3 系统基础支撑功能模块 29
5.3.1 跨域配置代码与安全说明 29
5.3.2 购物车数据传输对象实现 30
5.3.3 会员等级配置实体实现 30
6 系统测试 31
6.1 测试目的和策略 31
6.2 测试过程 31
6.2.1 管理端测试 31
6.2.2 用户端测试 32
6.3 测试总结 33
7 总 结 34
7.1 系统研发总结 34
7.2 局限性与后续优化 34
参考文献 36
致谢 38

1 绪论

1.1 研究背景

就目前而言，国内的体育用品消费市场是正向专业化还有精细化方向发展的，很多运动爱好者去选购体育用品时，对商品的材质或者适用场地这些参数的要求相对比以前更高了[1]；但是，从某种角度来说，中小型体育零售商想要去自主开发出一套高可用的线上商城，基本都会面临技术门槛比较高、后期维护也很困难的问题。比较明显的是，传统的单体架构只要一遇到促销活动引起流量突然增大，响应超时甚至服务器宕机的情况就很容易出现，所以，采用前后端分离架构去开发一套轻量级而且容易部署的体育用品销售网站，在实际应用上是有比较强的价值的[2]。

1.2 现有系统对比分析

项目初期调研了耐克官方商城、迪卡侬线上商城等主流平台。这些头部平台页面精美、分类详尽，但其架构庞大，中小商家难以复用。其客服也依赖人工转接，无法在非工作时间提供精准推荐。本系统则聚焦“低成本部署”与“智能化答疑”，精简非核心业务后，利用 Deep Seek 大模型实现了 24 小时商品推荐与订单查询。这种轻量级方案更契合中小商家的实际运营需求[3]。

1.3 开发的目的和意义

开发目的：Spring Boot 框架好升级、耐用的体育用品销售系统，可实现商品展示搜索、购物查看与支付、后台运维等核心功能，同时融入 Deep Seek 接口提供的智能客服，解决传统体育用品网站用户交互体验差的问题例如没有客服及时响应，为体育用品线上商家提供一套新且稳如泰山的思路。

研究意义：从理论的层面来讲，本课题的做法算是给那种垂直类的体育用品电商系统开发，去补充了一些带有实践性质的案例，也顺带着把大模型技术真正在电商卖货场景下的应用边界给拓宽了一下；在实际应用当中，主要就是给那些在网上卖体育用品的商家，弄出了一套从功能上相对有点新意、用起来感觉也比较现代的系统方案。这套方案被实际落地之后，其实不仅能在一定程度上帮着商家去稍微提高一下用户的

浏览体验跟最终买单的转化率，让后台管起来没那么费劲，同时这也是在给其他那些做单一传统产业的同行，提供一种可以拿去参考的、往线上智能化方向去转型的代码实现路子。

1.4 开发内容

本课题围绕基于 Spring Boot 的体育用品销售系统展开全流程开发，核心内容包括：

（1）系统分析与环境搭建：可行性分析、功能需求分析与用例分析，搭建基于 Spring Boot3.x+MyBatis-Plus 的后端环境、Vue3+Vite+Element Plus 的前端环境，以及 MySQL8.x 数据库环境；

（2）核心功能开发：实现前台用户模块（注册登录、商品浏览、购物车、订单支付、智能客服和会员成长等）与后台管理模块（商品管理、订单管理、公告管理、用户需求管理、会员管理和权限管理等）的全流程功能；

（3）创新功能设计：接入 Deep Seek 接口变为智能客服；

（4）测试与优化：测试功能、性能、稳定性、用户体验等，针对影响用户使用的问题进行优化迭代，保障稳定运行。

2 关键技术介绍

本系统采用前后端分离架构。后端以 Spring Boot 3.x 和 MyBatis-Plus 操作数据库，前端用 Vue 3 构建页面。数据层选用 MySQL 8.x，配合 Druid 连接池保障读写稳定。客服模块接入 Deep Seek 模型，实现自动应答。整套技术方案在支撑商品销售、订单管理等核心业务之外，也为后续迭代留出了扩展空间。

2.1 后端核心技术

2.1.1 Spring Boot 特性与优势

Spring 应用的搭建过程。它内嵌 Tomcat 等 Web 容器，项目可直接以 JAR 包形式运行，省去单独部署应用服务器的步骤。版本管理方面，通过 spring-boot-starter-parent 统一管理依赖版本，减少了手动配置版本号的兼容性冲突。

2.1.2 MyBatis-Plus 数据持久化

本系统用 MyBatis-Plus 3.5.15 做持久层框架。它比原生 MyBatis 多了通用 CRUD 接口、条件构造器和分页插件，写数据访问代码时省去大量重复的 SQL。没选 JPA 或 Hibernate，主要因为 MyBatis-Plus 对 SQL 控制更直接，复杂查询好优化，启动也快，和 Spring Boot 集成没什么坑。

依赖方面，pom 文件里引了 mybatis-plus-spring-boot4-starter，这个是针对 Spring Boot 4.x 的版本，解决了旧版 starter 的兼容问题。同时加了 mybatis-plus-jsqlparser，用来支撑分页和复杂 SQL 解析，两个依赖版本都定在 3.5.15。连接池用阿里的 Druid 1.2.23，通过 druid-spring-boot-3-starter 自动配置，带连接监控和慢查询检测功能。

2.1.3 MySQL 数据库技术

系统选用 MySQL 8.x 执行数据持久化，通过 RDBMS 建立器材 SKU 与订单流水间的逻辑关联。利用其事务隔离机制处理库存对冲业务，保证交易过程的数据准确。针对高并发场景下的连接压力，配置 Druid 连接池负责连接调度与状态监控。数据备份则直接调用数据库自带工具执行，确保存储架构稳定[3]

2.2 前端核心技术

2.2.1Vue3 组合式 API

本项目前端统一用 Vue 3 的<script setup>语法糖，省去手动 return 变量和方法的步骤。响应式数据用 ref 定义，比如首页的商品列表和轮播图都声明为 ref 对象。生命周期用 onMounted，组件挂载后自动请求数据。路由参数通过 useRoute 获取，分类切换用 watch 监听 query 变化，参数一变就重新加载，页面不刷新。组合式 API 把数据加载、格式化等相关逻辑收在一处，比选项式 API 结构更紧凑。

2.2.2Vite 构建与组件化开发

前端用 Vite 做构建工具。开发时基于原生 ES 模块启动，冷启动和热更新都及时完成及改完代码页面立刻生效。架构上采用组件化开发，页面拆成 Header、ProductGrid 等独立组件，每个组件管自己的逻辑和视图。配合 Vue 3 的组合式 API，组件复用起来方便。加新页面或改现有模块都不用大动结构，维护成本可控。

2.2.3Axios 前后端交互

前后端数据交互用 Axios 处理。项目里封装了两个请求实例，frontRequest.js 负责用户端的 API 请求，adminRequest.js 负责后台管理端的请求，两个实例各自配了 baseURL 和拦截器，Token 认证和错误处理都统一收在拦截器里。

组件里用 async/await 发异步请求，比如 Home.vue 加载商品和轮播图数据时，用 try-catch 包住请求做异常捕获。封装之后，业务代码里不用每次手写请求头和错误处理逻辑，调用封装好的方法就行，网络请求那部分看起来简洁，改起来也方便。

2.3 安全与规范标准遵循

本系统注重信息安全与规范落地。用户登录用 Session 维持会话状态，鉴权完成后清空内存中的明文密码，防止敏感数据残留。接口通信层面，所有前后端请求都遵循 RESTful API 规范。增删改查操作对应 POST、DELETE、PUT、GET 四种标准 HTTP 方法，接口语义明确，代码可读性也更好。

3 系统分析

3.1 需求分析

本项目分别从用户端和管理端进行业务逻辑设计，涵盖商品销售、订单流转及后台管控等环节[5]。

用户端主要服务运动爱好者的选购需求。前台提供首页推荐、关键词检索和商品详情展示。购物车用 VO 对象封装数据，保证加购到支付各环节的安全。订单模块支持实时物流查询和售后反馈。系统集成了 Deep Seek 接口，提供全天候在线导购。

管理端侧重运营效率和库存控制。管理员有商品维护权限，负责品牌分类、定价和素材更新。订单与售后审核模块监控交易状态，执行物流节点分发。后台提供用户画像维护与公告发布两项功能。管理人员可按用户活跃度调整运营内容。

3.2 技术可行性

后端用 JDK 17 和 Spring Boot 框架，能保证数据库并发事务高效执行。前端靠 Vue 3 的组件化开发和 Vite 工具缩短页面加载时间。AI 模块通过标准 RESTful API 接入，通信协议统一，接口调用可控。这套技术栈选型成熟，部署和维护成本都不高，完全符合工程开发要求[6]：

（1）后端：搭配 JDK17 的 Spring Boot 框架在内存管理和执行效率上具备优势能稳定承载数据库并发事务。

（2）前端：“Vue3+Vite”的组合提供了热更新与组件化能力，确保前台现代化视觉效果和后台管理面板的丝滑呈现。

（3）AI 集成：Deep Seek 大语言模型通过标准化的 RESTful 协议与后端连接，技术接口透明，调用链路清晰，具备极强的工程落地可行性。

3.3 运行可行性

系统的运行设计遵循“易用性”与“自动化”原则：

（1）人机交互界面：基于 Element Plus 标准库设计，符合主流体育用品销售操作逻辑，降低了用户的学习成本。

（2）运维成本：通过集成 Deep Seek AI，系统能够自动处理业务咨询，极大降低

了人工客服的运营成本。

（3）部署环境：体育用品销售系统对服务器硬件配置要求适中，支持容器化部署，具备良好的环境迁移能力。

3.4 经济可行性

项目在开发与维护成本上具有优势，契合中小型体育用品商户的预算规划。采用了开源技术栈不仅规避了的商业软件授权费，且开发工具 IntelliJ IDEA 低成本特性降低了前期的资金投入。系统对服务器的硬件配置要求适中，无须购置高性能集群对中小型体育用品商户来说，这就进一步压缩了开支。此外，集成 Deep Seek 大模型客服功能替代了部分人工客服需求，在有效提升响应速度的同时，兼顾夜间提供服务。

综上所述，无论是建设还是日常运营，项目均能以较低的经济投入达到预期商业目标，具备经济可行性。

3.5 操作可行性

系统在交互设计上区分了用户端和管理端的操作需求。用户端采用常见的电商页面布局，商品检索、购物车结算等操作符合一般用户的浏览习惯，上手没有额外学习负担。后台管理面板把高频操作放在首页，比如商品上下架、库存调整、公告和轮播图发布等，常用功能入口明显，管理员不用反复翻菜单。

业务操作的结果有即时反馈，订单支付后状态实时更新，售后申请的审批进度也可跟踪，用户和管理员都能看到当前走到哪一步。界面基于 Element Plus 组件库搭建，视觉风格统一，布局和交互方式符合常规后台系统的操作习惯，方便后续推广使用。

4 系统设计

4.1 体育用品销售网站功能结构设计

系统功能按角色划分为用户端和管理端两大模块[9]。

用户端面向消费者，负责商品展示和购买交互，涵盖商品检索、购物车、订单确认、会员服务和智能导购等功能，聚焦选购流程的便利性。管理端面向运营人员，负责后台管控，涵盖商品上下架与库存调整、订单流转跟踪、售后审核处置和公告发布等功能，聚焦运营管理的精准度。

两个模块通过数据接口连接，用户端产生的操作数据同步到管理端，管理端的配置变更也实时反映到用户端。两者在逻辑上解耦，在数据上互通。系统总体功能结构图如图 4.1 所示：

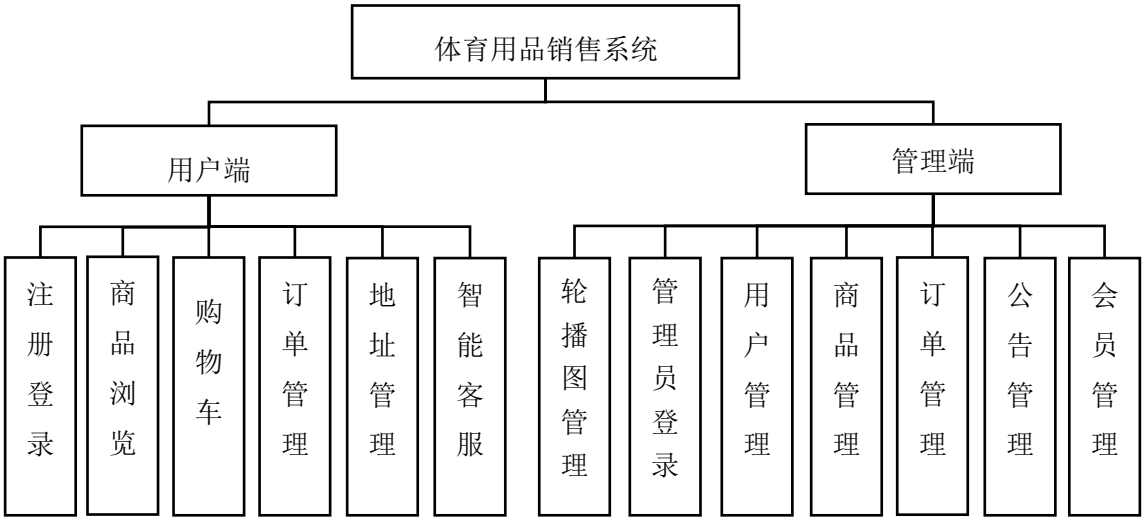


图 4.1 系统总体功能结构图

4.2 体育用品销售网站数据库设计

本系统用 MySQL 做数据存储。数据表按第三范式来建，能减少冗余[10]。表结构规范了，高并发写入时数据一致性就更有保障。

4.2.1 数据库设计

概念模型把用户和管理员作为参与主体，体育器材抽象为商品实体。业务逻辑上，加购数据先放进购物车表缓冲。用户点结算后，生成正式订单。各物理表用外键建立关联约束，保证存储结构逻辑一致。

（1）管理员实体用于后台身份验证，核心是存储账号和登录凭证。数据库设计上，账号字段设置唯一约束防止重复注册。这措施为后台权限分配提供了安全基础。管理员实体图如下图 4.2 所示：

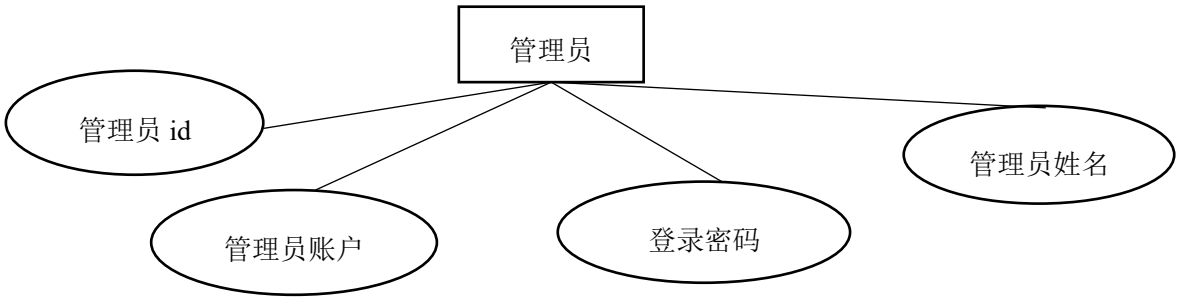


图 4.2 管理员实体图

（2）用户实体存储消费者的身份信息、账户状态和会员数据。表中记录用户名和登录密码作为基础凭证，同时存手机号、邮箱和头像等标识信息。会员成长方面，用 experience 字段累计消费产生的经验值，current_level 记录当前等级，等级随消费动态变化。status 字段控制账户启用或禁用，方便处理异常账号。login_days 和 last_login_date 记录登录频次和末次活跃日期，为运营分析提供数据参考如图 4.3 所示：

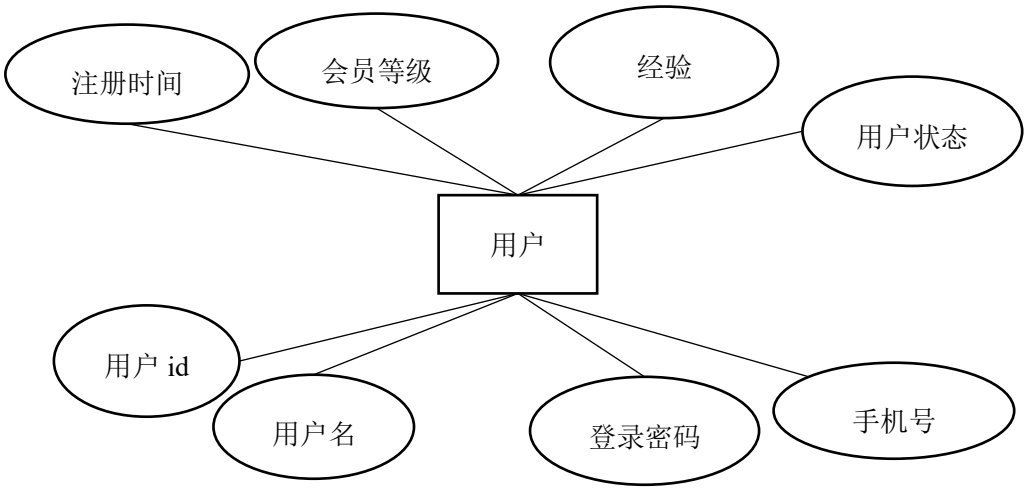


图 4.3 用户实体图

（3）商品实体存放器材的基础信息和销售数据。字段设计上，除了商品名称和所属分类，原价和售价分开记录，方便后续打折促销时做对比。库存数量和累计销量分别用 stock 和 sales_count 字段存储，商品上下架状态也做了标记，前端展示时自动过滤下架商品，后台也能据此跟踪库存和销售情况。商品具体如图 4.4 所示：

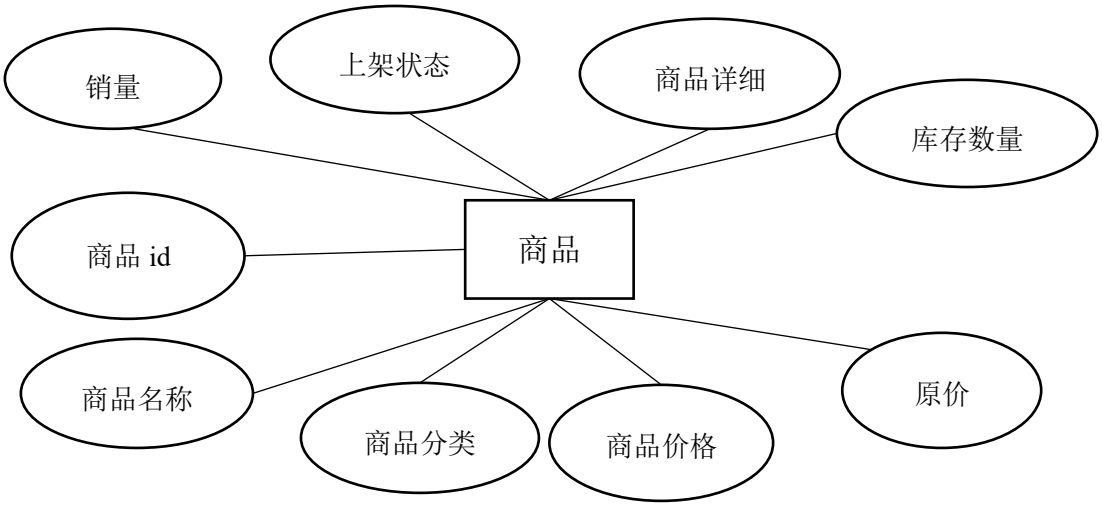


图 4.4 商品实体图

（4）购物车作为用户与商品之间的中间桥梁，用于暂存用户的待购器材信息。其核心逻辑在于通过 user_id 关联目标用户，通过 product_id 锁定具体商品，并利用 quantity 字段记录选购数量。这种设计支持用户的多轮添加，为后续订单生成的批量处理提供基础。购物车见图 4.5：

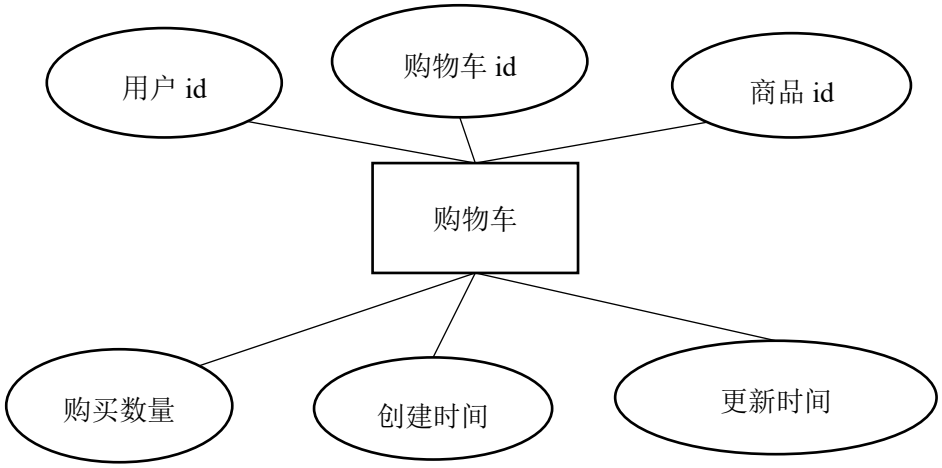


图 4.5 购物车配置实体图

（5）订单实体保存用户购买后的交易记录。字段层面，通过用户 ID 和订单编号关联下单用户，总金额和实付金额分开存储，便于后续财务对账。收货地址、创建时间、支付时间和发货时间等时间节点与订单状态一并记录，用于跟踪支付进度和物流情况。订单图如 4.6 所示：

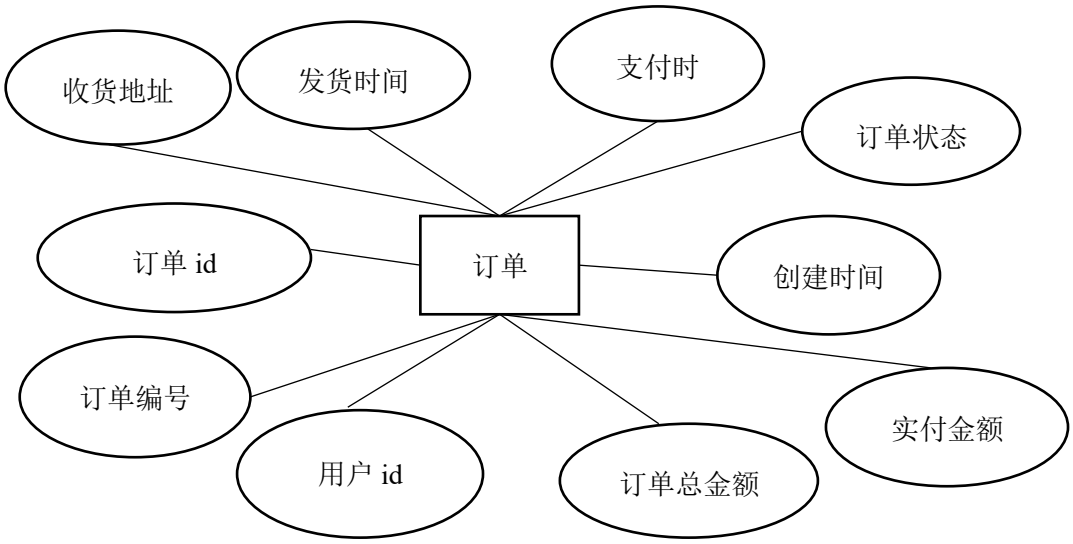


图 4.6 订单实体图

4.2.2 数据库表设计

下面只介绍几张核心数据表的设计思路。

管理员表存后台账号和凭证。id 是自增主键。manager 字段记录账号名，配了唯一索引，防重名。password 字段存登录密码。系统靠这张表做高权限账户校验，防止后台数据被越权篡改。字段定义详见表 4.1：

表 4.1 管理员表（manager）

字段名称	数据类型	主键	外键	允许空	默认值	说明
id	int	是	否	否	自增	管理员 ID
manager	varchar	否	否	否	无	管理员账号
password	varchar	否	否	否	无	登录密码

用户表主要用于存储账号的基础信息及会员成长数据。在账号管理层面，该表记录用户名、密码、邮箱及手机号等凭证，系统通过“status”字段对账号执行启用或禁用操作，确保管理员能够高效处理异常及违规账户。

会员成长方面，experience 字段累计每次消费产生的经验值，经验值达到对应阈值后自动更新 current_level 字段，实现等级晋升。is_vip 和 vip_expire_time 标记会员身份及有效期。此外，login_days 记录连续登录天数，last_login_date 记录末次活跃日期，这两个字段为运营人员分析用户活跃度和制定营销策略提供数据参考。具体字段

分布见表 4.2：

表 4.2 用户表（user）

字段名称	数据类型	主键	外键	允许空	默认值	说明
id	bigint	是	否	否	自增	用户 ID
username	varchar	否	否	否	无	用户名
password	varchar	否	否	否	无	登录密码
email	varchar	否	否	是	NULL	邮箱
phone	varchar	否	否	是	NULL	手机号
avatar	varchar	否	否	是	NULL	头像
status	tinyint	否	否	否	1	状态：1 正常，0 禁用
created_at	datetime	否	否	否	当前时间	注册时间
updated_at	datetime	否	否	否	当前时间	更新时间
is_vip	tinyint	否	否	是	0	是否 VIP：0 否，1 是
vip_expire_time	datetime	否	否	是	NULL	VIP 过期时间
experience	bigint	否	否	是	0	经验值
current_level	varchar	否	否	是	灵胎	当前等级
login_days	int	否	否	是	0	连续登录天数
last_login_date	date	否	否	是	NULL	最后登录日期

商品表用于存储器材基础信息。利用 category_id 关联所属分类，通过 price 和 original_price 字段展示售价及原价。stock 记录实时库存，sales_count 则统计累计销量。商品上架或下架状态由 status 枚举字段控制，处于下架状态的器材前端会自动屏蔽。表 4.3 列出了各字段的具体定义：

表 4.3 商品表（product）

字段名称	数据类型	主键	外键	允许空	默认值	说明
id	bigint	是	否	否	自增	商品 ID
category_id	bigint	否	是	否	无	分类 ID
name	varchar	否	否	否	无	商品名称
description	text	否	否	是	NULL	商品描述
price	decimal	否	否	否	无	售价
original_price	decimal	否	否	是	NULL	原价
stock	int	否	否	否	0	库存
sales_count	int	否	否	否	0	销量
main_image	varchar	否	否	否	无	主图地址
status	enum	否	否	否	上架	商品状态
created_at	datetime	否	否	否	当前时间	创建时间
updated_at	datetime	否	否	否	当前时间	更新时间

购物车表用来暂存用户挑中的器材。id 是自增主键。user_id 和 product_id 分别关联用户和商品。quantity 记录购买件数。created_at 和 updated_at 两个时间戳用来跟踪记录的生成和变动。字段定义详见表 4.4：

表 4.4 购物车表（cart）

字段名称	数据类型	主键	外键	允许空	默认值	说明
id	bigint	是	否	否	自增	购物车项 ID
user_id	bigint	否	是	否	—	用户 ID
product_id	bigint	否	是	否	—	商品 ID
quantity	int	否	否	否	1	购买数量
created_at	datetime	否	否	否	当前时间	创建时间
updated_at	datetime	否	否	否	当前时间	更新时间

订单表包含唯一的订单编号、关联的用户 ID、订单总金额、优惠金额及实付金

额等核心财务字段，创建时间默认为下单时的当前时间。字段定义见表 4.5：

表 4.5 订单表（order）

字段名称	数据类型	主键	外键	允许空	默认值	说明
id	bigint	是	否	否	自增	订单 ID
order_no	varchar	否	否	否	无	订单编号
user_id	bigint	否	是	否	无	用户 ID
total_amount	decimal	否	否	否	无	订单总金额
discount_amount	decimal	否	否	是	0.00	优惠金额
pay_amount	decimal	否	否	否	无	实付金额
address_snapshot	text	否	否	否	无	地址快照
status	enum	否	否	否	待支付	订单状态
payment_method	enum	否	否	是	NULL	支付方式
paid_at	datetime	否	否	是	NULL	支付时间
created_at	datetime	否	否	否	当前时间	创建时间
remark	varchar	否	否	是	NULL	订单备注

5 系统实现

5.1 管理员模块

管理端统一管控核心业务数据。权限校验会拦截未授权访问，管理员通过身份验证后，才能操作器材类目和订单数据。这个管理员模块除了基础维护，还管订单售后。保证业务流程顺利满足用户要求[11]。界面效果见图 5.1：



图 5.1 管理员实现图

5.1.1 管理员登录的实现

后端接口拿到前端传来的账号密码，去数据库匹配。核心代码如下：

```
@PostMapping("/login")
    QueryWrapper<Manager> wrapper = new QueryWrapper<>();
    wrapper.eq("manager", loginParam.getName().trim());
    wrapper.eq("password", loginParam.getPwd().trim());

    Manager manager = managerService.getOne(wrapper);

    if (manager == null) {
        return R.fail("登录失败，用户名或密码错误");
}
```

界面如图 5.2 所示：



图 5.2 用户登录实现图

5.1.2 商品管理的实现

管理端有器材数据维护入口。运营人员能在线录基础资料，实时改参数，切换上下架状态。热销商品支持快速补仓。后台同步汇总用户浏览数据，帮库存调配和经营分析做参考。核心代码如下：

```
@GetMapping("/list")
public R list(@RequestParam Map<String, Object> params) {
    int page = params.get("page") == null ? 1 : Integer.parseInt(params.get("page").toString());
    int size = params.get("size") == null ? 10 : Integer.parseInt(params.get("size").toString());
    Page<Product> pageParam = new Page<>(page, size);
    QueryWrapper<Product> wrapper = new QueryWrapper<>();
    if (params.get("name") != null) {
        wrapper.like("name", params.get("name")); }
    if (params.get("categoryId") != null) {
        wrapper.eq("category_id", params.get("categoryId"));}
    wrapper.orderByDesc("id");
    Page<Product> result = productService.page(pageParam, wrapper);
    Map<String, Object> data = new HashMap<>();
    data.put("list", result.getRecords());
    data.put("total", result.getTotal());
    return R.ok(data);}
@PostMapping("/add")
public R add(@RequestBody Product product) {
    return productService.save(product) ? R.ok() : R.fail();}
@PostMapping("/update")
```

```
public R update(@RequestBody Product product) {
    return productService.updateById(product) ? R.ok() : R.fail();}
```

界面见图 5.3：

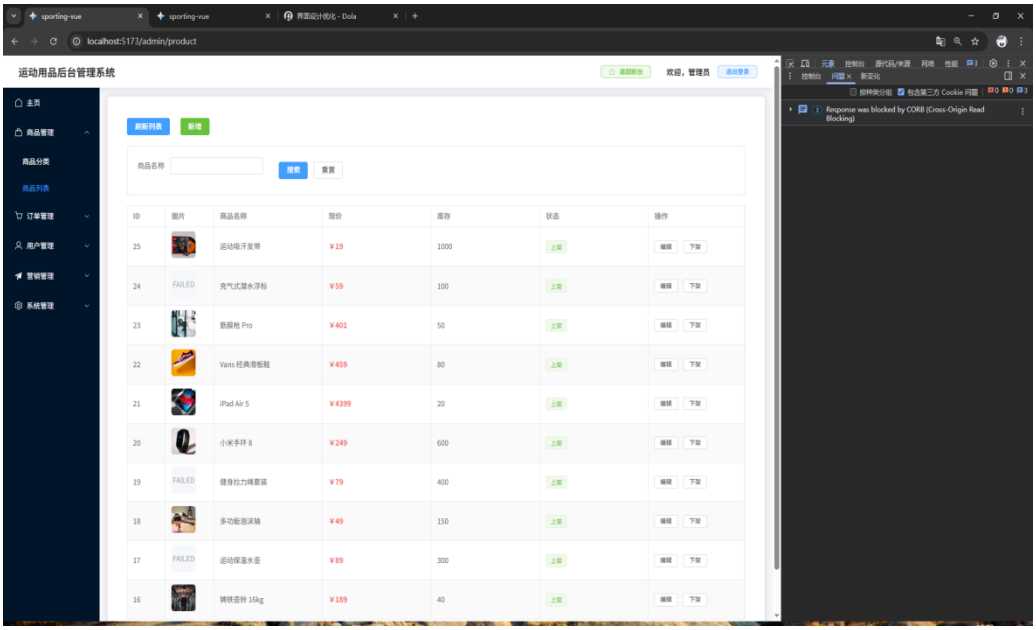


图 5.3 商品管理实现图

5.1.3 用户管理的实现

用户管理模块的后端代码主要实现了分页查询、单条详情查看和删除等接口。分页查询时前端传入参数，后端用 MyBatis-Plus 的 Page 对象封装，查到数据后把记录列表和总数一起返回。详情接口查不到用户时返回“用户不存在”，查到后在返回前把 password 字段置空，避免敏感数据暴露。删除接口直接调用 removeById 完成。核心代码如下：

```
@GetMapping("/list")
public R list(@RequestParam(defaultValue = "1") Integer page,
              @RequestParam(defaultValue = "10") Integer size) {
    Page<User> pageParam = new Page<>(page, size);
    Page<User> result = userService.page(pageParam);
    Map<String, Object> data = new HashMap<>();
    data.put("list", result.getRecords());
    data.put("total", result.getTotal());
    return R.ok(data);}
@GetMapping("/{id}")
public R getInfo(@PathVariable Long id) {
```

```
        User user = userService.getById(id);
        if (user == null) {
            return R.fail("用户不存在"); }
        user.setPassword(null);
        return R.ok(user);}
    @DeleteMapping("/{id}")
    public R delete(@PathVariable Long id) {
        userService.removeById(id);
        return R.ok("删除成功");}
```

界面见图 5.4：

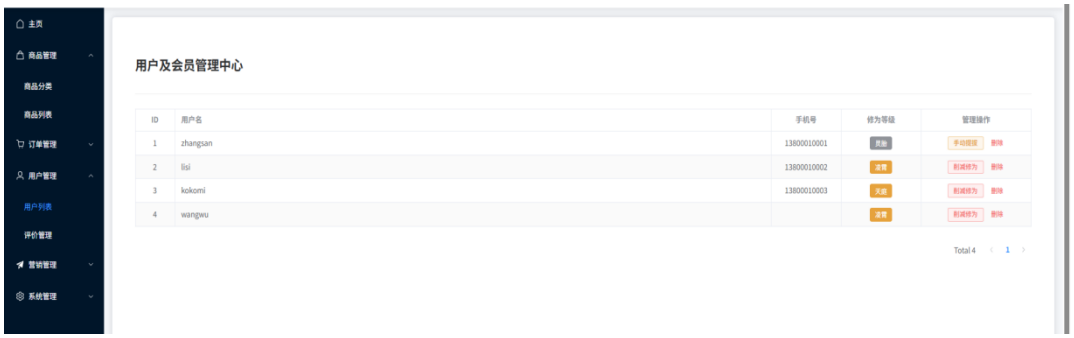


图 5.4 用户管理实现图

5.1.4 订单管理的实现

订单管理模块提供分页查询和删除两个接口。分页查询接收 page 和 size 参数，后端用 MyBatis-Plus 的 Page 对象封装后查库，返回记录列表和总条数。管理员可以看到每笔订单的状态、金额和物流信息，实时跟踪器材去向。删除接口根据订单 ID 直接调用 removeById 清理数据，用于处理异常单据。核心代码如下：

```
    @GetMapping("/list")
    public R list(@RequestParam(defaultValue = "1") Integer page,
                  @RequestParam(defaultValue = "10") Integer size) {
        Page<Order> p = new Page<>(page, size);
        Page<Order> result = orderService.page(p);
        Map<String, Object> data = new HashMap<>();
        data.put("list", result.getRecords());
        data.put("total", result.getTotal());
        return R.ok(data);}
    @DeleteMapping("/{id}")
    public R delete(@PathVariable Long id) {
```

```
        orderService.removeById(id);
        return R.ok("删除成功");}
```

界面见图 5.5：

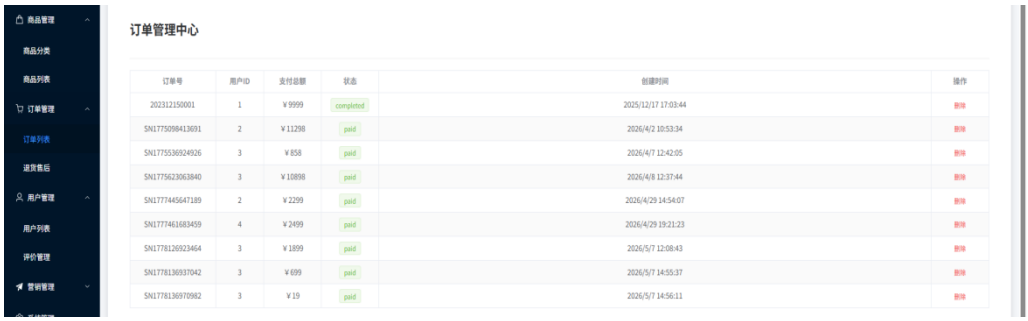


图 5.5 订单管理实现图

5.1.5 管理员受理退换货实现

退换货申请统一汇总到后台售后模块，管理员通过分页列表查看，列表接口前端传入 page 和 size 参数，后端返回记录和总条数。审核时调用 approve 接口，根据退换货 ID 查出对应单据，存在就把 status 字段置为 1 表示已通过，然后更新保存；查不到单据就返回"单据不存在"。这个模块把售后处理流程收拢到后台，管理员查验凭证后手动执行审核即可。核心代码如下

```
@GetMapping("/list")
public R list(@RequestParam(defaultValue = "1") Integer page,
              @RequestParam(defaultValue = "10") Integer size) {
    Page<ReturnRequest> pageParam = new Page<>(page, size);
    Page<ReturnRequest> result = returnRequestService.page(pageParam);
    Map<String, Object> data = new HashMap<>();
    data.put("list", result.getRecords());
    data.put("total", result.getTotal());
    return R.ok(data);}
@PostMapping("/approve/{id}")
public R approve(@PathVariable Long id) {
    ReturnRequest req = returnRequestService.getById(id);
    if (req != null) {
        req.setStatus(1); // 1: 已通过
        returnRequestService.updateById(req);
        return R.ok("审核已通过"); }
    return R.fail("单据不存在");}
```

界面见图 5.6：

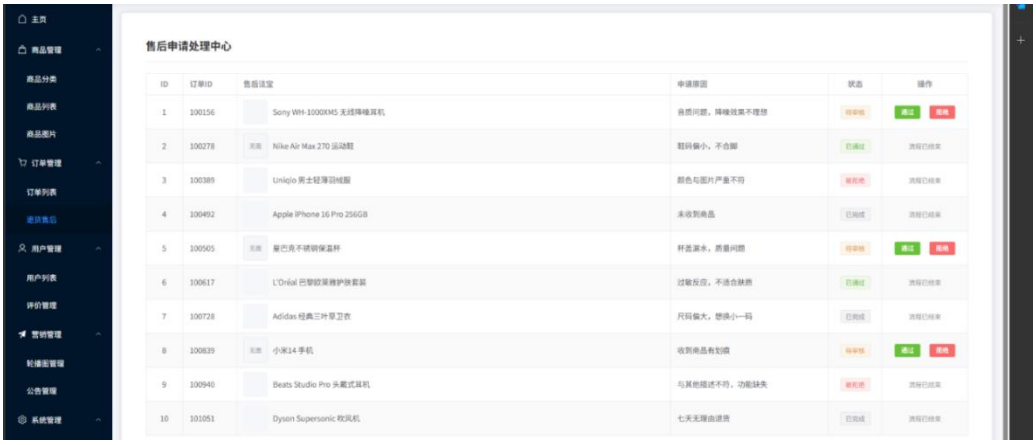


图 5.6 退换货实现图

5.1.6 轮播图管理实现

轮播图模块后端提供了列表查询、新增、删除和图片上传四个接口。列表接口直接查全部 Banner 记录返回；新增和删除接口调用 Service 层方法，成功返回对应提示，失败返回错误信息。上传接口处理 MultipartFile 文件，先判断是否为空，然后检查存放目录是否存在，不存在就创建。文件名用当前毫秒时间戳加原始文件名拼接，防止新老图片重名。上传后拼接完整的图片 URL 返回给前端。核心代码如下：

```
@Autowired
private BannerService bannerService;
private static final String UPLOAD_DIR = "D:/code/sporting-vue/public/uploads/";
@GetMapping("/list")
public R list() {
    List<Banner> banners = bannerService.getAllBanners();
    return R.ok(banners);}
@PostMapping("/add")
public R add(@RequestBody Banner banner) {
    return bannerService.saveBanner(banner) ? R.ok("新增成功") : R.fail("新增失败");}
@DeleteMapping("/delete/{id}")
public R delete(@PathVariable Long id) {
    return bannerService.removeBanner(id) ? R.ok("删除成功") : R.fail("删除失败");}
@PostMapping("/upload")
public R uploadImage(@RequestParam("file") MultipartFile file) throws IOException {
    if (file.isEmpty()) return R.fail("上传文件不能为空");
```

```
File uploadDirectory = new File(UPLOAD_DIR);
if (!uploadDirectory.exists()) {
    uploadDirectory.mkdirs();}
String originalName = file.getOriginalFilename();
String fileName = System.currentTimeMillis() + "_" + originalName;
File dest = new File(UPLOAD_DIR + fileName);
file.transferTo(dest);
String imageUrl = "http://localhost:80/uploads/" + fileName;
return R.ok("上传成功", imageUrl);}
```

界面见图 5.7



图 5.7 轮播图管理实现图

5.1.7 公告管理实现

公告管理模块提供了列表查询、新增、修改和删除四个接口。列表查询按 publish_at 字段倒序排列，保证最新发布的公告排在最前面，方便运营人员查看。新增接口接收前端传来的公告内容直接保存；修改接口根据传回的公告对象更新对应记录；删除接口按公告 ID 移除数据。整个模块把公告的增删改查都收拢在后台，运营人员可以随时更新促销信息和维护通知，前端拿到最新数据后同步展示。核心代码如下：

```
@GetMapping("/list")
public R list() {
    List<Announcement> list = announcementService.list(
            new QueryWrapper<Announcement>().orderByDesc("publish_at"));
    return R.ok().setData(list);}
```

```
@PostMapping("/add")
public R add(@RequestBody Announcement announcement) {
    announcementService.save(announcement);
    return R.ok();}
@PutMapping("/update")
public R update(@RequestBody Announcement announcement) {
    announcementService.updateById(announcement);
    return R.ok();}
@DeleteMapping("/delete/{id}")
public R delete(@PathVariable Integer id) {
    announcementService.removeById(id);
    return R.ok();}
```

界面见图 5.8：

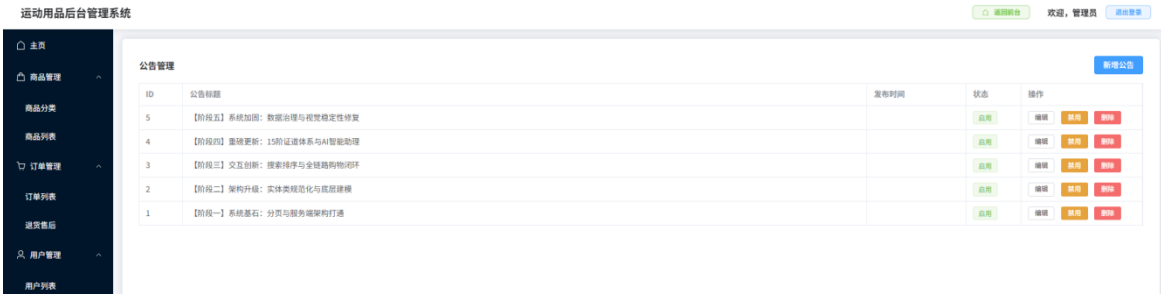


图 5.8 公告管理实现图

5.2 前台用户模块

用户端主负责商品展示和交互。首页用轮播图展示热销器材。模糊搜索方便找商品。个人中心集成了地址修改、资料管理和订单记录查询。用户在这就能发起售后退款。会员升级逻辑由后台自动算好执行。前台界面效果见图 5.9 和 5.10：

界面见图 5.9：

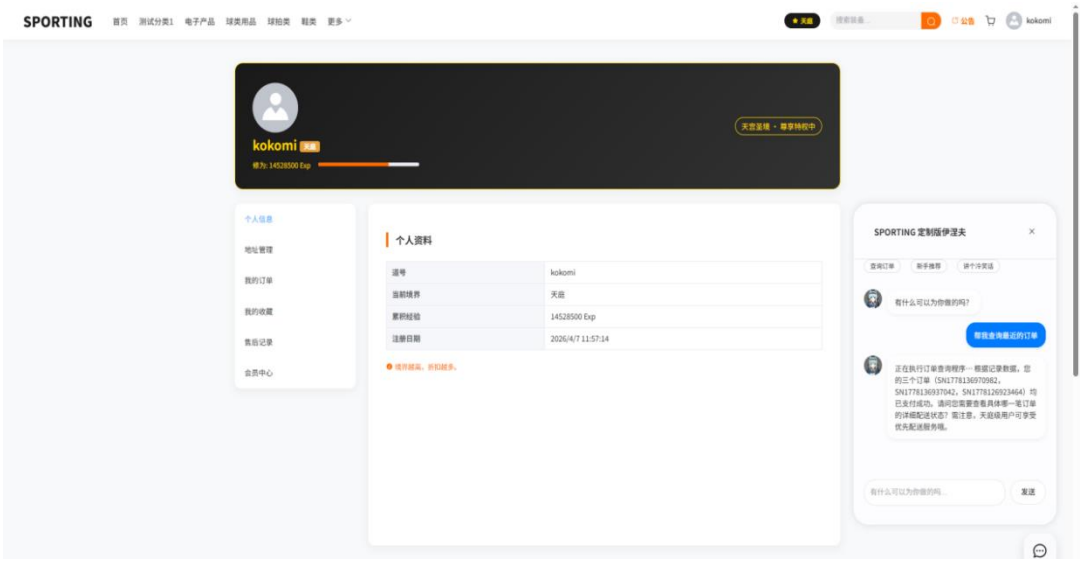


图 5.9 用户中心实现图

界面见图 5.10：

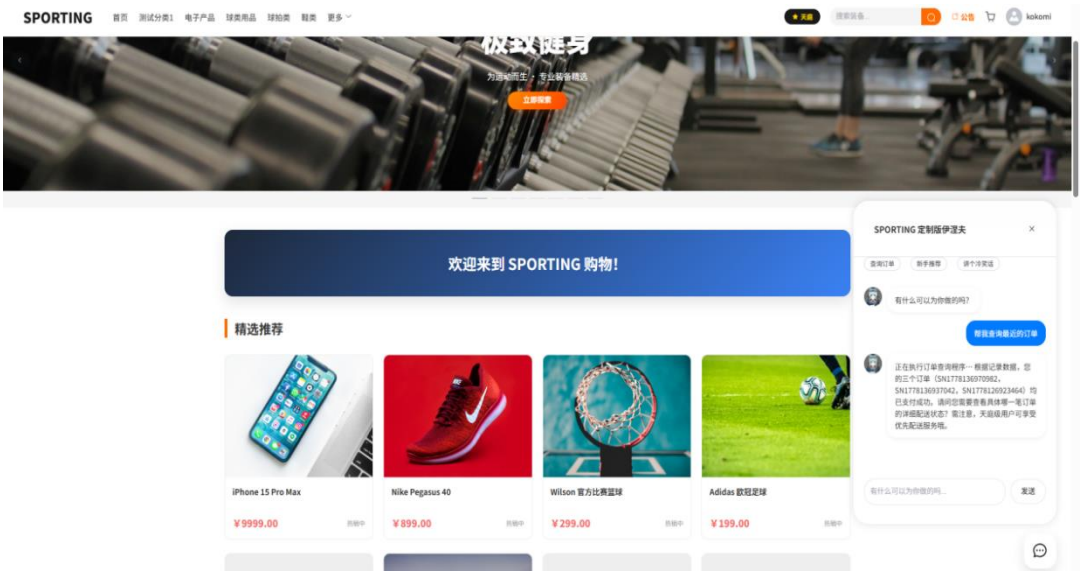


图 5.10 前台用户实现图

5.2.1 用户注册和登录实现

登录接口先校验前端传来的用户名和密码是否为空，空则直接返回提示。不为空就用 LambdaQueryWrapper 构造查询条件，到 user 表里匹配用户名和密码，查不到就返回“账号或密码错误”。查到后把用户 ID 存入 Session，再将密码字段置空再返回给前端，避免密码在内存中留存。

注册接口先检查用户名是否为空或纯空格，然后校验密码长度是否小于 6 位，两项校验不通过就返回对应提示。接着查库看用户名是否已被占用，存在就提示换一个。

两个接口配合完成了用户身份的注册和登录校验。核心代码如下：

```
@PostMapping("/login")
public R login(@RequestBody User user, HttpServletRequest request) {
    if (user.getUsername() == null || user.getPassword() == null) {
        return R.fail("用户名和密码不能为空");}
    User dbUser = userService.getOne(
            new LambdaQueryWrapper<User>()
                    .eq(User::getUsername, user.getUsername())
                    .eq(User::getPassword, user.getPassword()));
    if (dbUser == null) { return R.fail("账号或密码错误"); }
    request.getSession().setAttribute("userId", dbUser.getId());
    dbUser.setPassword(null); return R.ok(dbUser);}
@PostMapping("/register")
public R register(@RequestBody User user) {
    if (user.getUsername() == null || user.getUsername().trim().isEmpty()) {
        return R.fail("用户名不能为空");}
    if (user.getPassword() == null || user.getPassword().length() < 6) {
        return R.fail("密码安全性过低（至少 6 位）");}
    User existingUser = userService.getOne(
    new LambdaQueryWrapper<User>()
        .eq(User::getUsername, user.getUsername()));
    if (existingUser != null) {return R.fail("该名号已被其他道友占用，请换一个");}
```

界面见图 5.11：

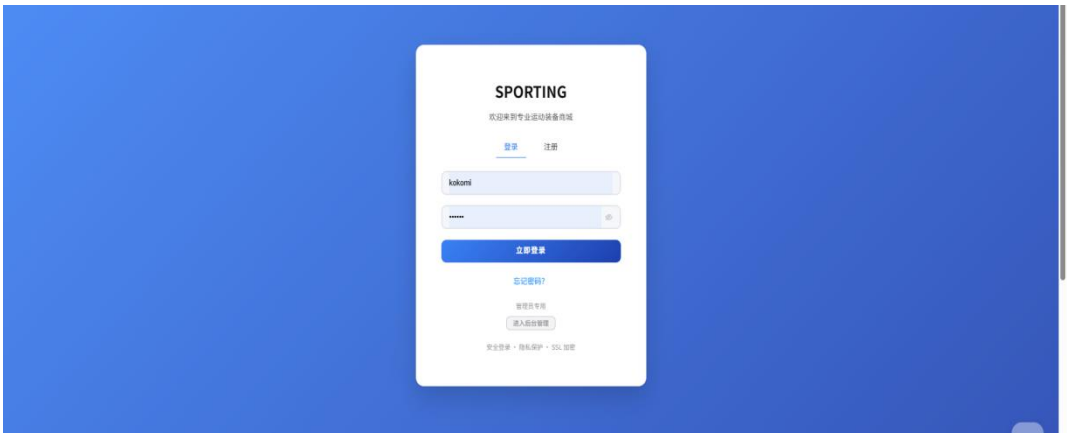


图 5.11 用户注册与登录实现图

5.2.2 商品浏览和搜索功能实现

商品浏览和搜索主要靠两个接口。详情接口根据商品 ID 查库，查到后先判断状

态是不是“on_sale”，下架或不存在都返回对应提示，只有上架商品才返回详情数据给前端。列表接口接收分类 ID、关键词和排序方式三个可选参数，后端用 LambdaQueryWrapper 动态拼接查询条件。如果传了分类 ID 就按分类筛选，传了关键词就按商品名模糊匹配，同时强制过滤掉下架状态的商品，确保前端只展示可售器材。排序参数用来控制按价格或创建时间排列。核心代码如下：

```
@GetMapping("/{id}")
public R detail(@PathVariable Long id) {
    Product product = productService.getById(id);
    if (product == null || !"on_sale".equals(product.getStatus())) {
        return R.fail("商品不存在或已下架");}return R.ok(product);}
@GetMapping("/list")
public R list(@RequestParam(required = false) Long categoryId,
              @RequestParam(required = false) String keyword,
              @RequestParam(required = false) String sort) {
    LambdaQueryWrapper<Product> wrapper = new LambdaQueryWrapper<>();
```

界面如图 5.12：

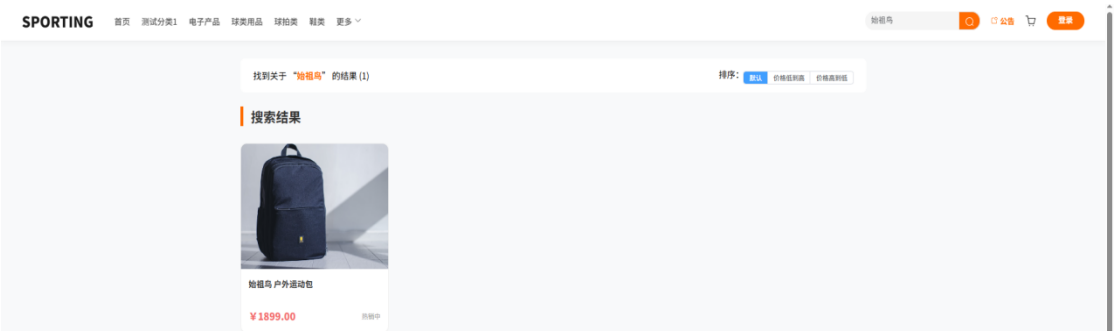


图 5.12 搜索实现图

界面如图 5.13：



图 5.13 商品浏览实现图

5.2.3 购物车功能实现

购物车模块暂存用户挑的器材。做了唯一性校验，同一用户加同一商品只留一条记录。重复加购就自动更新数量。查询时后台关联商品主表，拿最新单价算小计。数据封装到 CartVO 视图对象，下发给前端，包含商品名称、图片和金额。核心代码如下：

```
@PostMapping("/add")
public R add(@RequestBody Cart cart) {
    Cart exist = cartService.lambdaQuery()
            .eq(Cart::getUserId, cart.getUserId())
            .eq(Cart::getProductId, cart.getProductId()) .one();
    if (exist != null) {
        exist.setQuantity(exist.getQuantity() + cart.getQuantity());
        cartService.updateById(exist);
        return R.ok("数量已更新");  }
    cartService.save(cart);
    return R.ok("已加入购物车");}
@DeleteMapping("/delete/{id}")
public R delete(@PathVariable Long id) {
    cartService.removeById(id); return R.ok();}
```

界面如图 5.14：

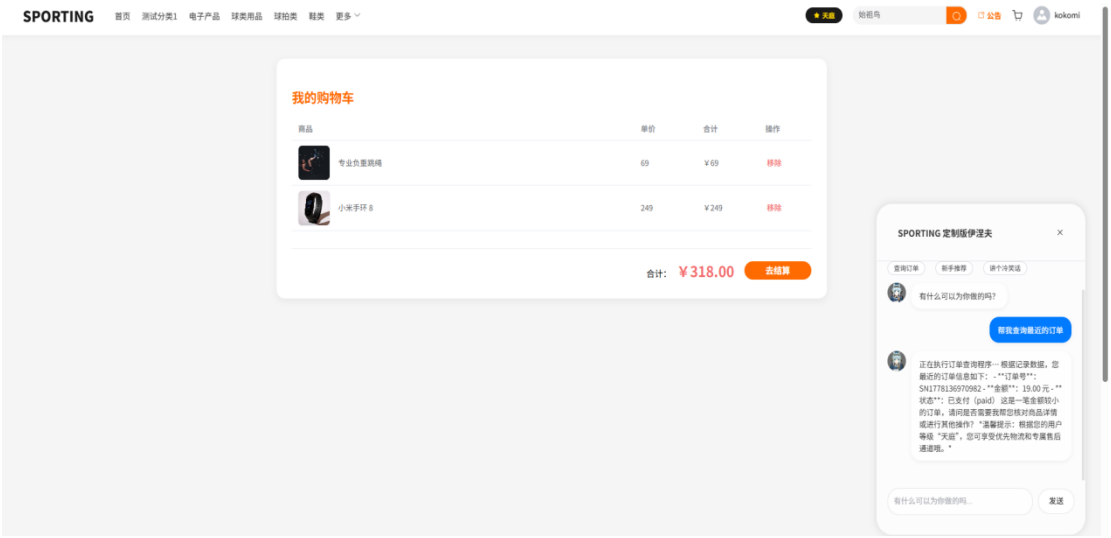


图 5.14 购物车实现图

5.2.4 订单下单实现

下单模块把购物车存根转成正式交易记录。/preview 接口汇总商品信息，算出总额，匹配地址。用户确认提交后，后端开启事务一口气完成库存预扣、订单生成和购物车清理。列表与详情接口用来查状态和追溯明细。核心代码如下：

```
@GetMapping("/preview")
public R preview(@RequestParam Long userId) {
    List<Cart> carts = cartService.lambdaQuery().eq(Cart::getUserId, userId).list();
    List<CartVO> items = new ArrayList<>();
    BigDecimal total = BigDecimal.ZERO;
    for (Cart c : carts) {
        Product p = productService.getById(c.getProductId());
        if (p != null) {
            CartVO vo = new CartVO();
            vo.setProductId(p.getId());
            vo.setProductName(p.getName());
            vo.setProductImage(p.getMainImage());
            vo.setPrice(p.getPrice());
            vo.setQuantity(c.getQuantity());
            BigDecimal sub = p.getPrice().multiply(new BigDecimal(c.getQuantity()));
            vo.setSubtotal(sub);
            total = total.add(sub);
            items.add(vo); }}
    Address addr = addressService.lambdaQuery().eq(Address::getUserId, userId).eq(Address::ge
tIsDefault, 1).one();
    Map<String, Object> res = new HashMap<>();
    res.put("items", items);
    res.put("totalAmount", total);
    res.put("address", addr != null ? addr : new HashMap<>());
    return R.ok(res);}
```

界面如图 5.15：

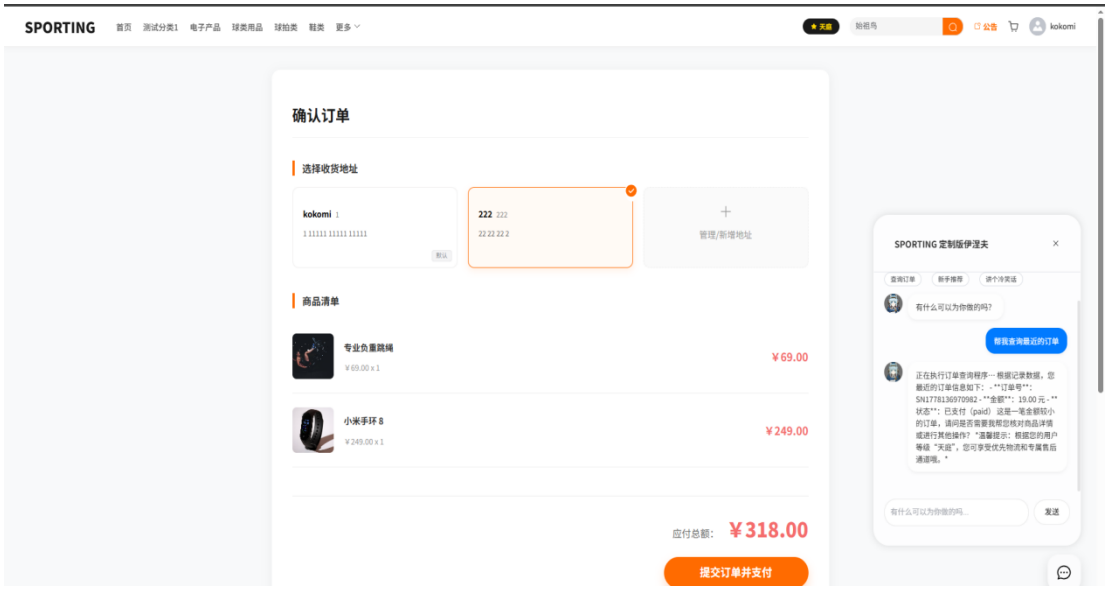


图 5.15 订单下单

5.2.5 支付功能实现

支付提交接口先校验 userId 和 addressId 两个必传参数，任一为空就直接返回提示。参数校验通过后，取出 userId 和 addressId，调用 orderService 的 createOrder 方法执行下单逻辑。下单成功返回订单 ID 给前端，失败则捕获异常并返回错误信息。NumberFormatException 对应参数类型转换错误，RuntimeException 对应业务逻辑中的异常，比如库存不足等情况。支付完成后，订单状态由“待支付”更新为“已支付”。前端展示支付选项和收款二维码，用户确认到账后，后端同步运行会员积分计算，按消费金额累加经验值。核心代码如下：

```
@PostMapping("/submit")
public R submit(@RequestBody Map<String, Object> params) {
    if (params.get("userId") == null || params.get("addressId") == null) {
        return R.fail("用户 ID 和地址 ID 不能为空");}
    try {
        Long userId = Long.valueOf(params.get("userId").toString());
        Long addressId = Long.valueOf(params.get("addressId").toString());
        Long orderId = orderService.createOrder(userId, addressId);
        Map<String, Object> result = new HashMap<>();
        result.put("orderId", orderId);
        return R.ok(result);
    } catch (NumberFormatException e) {
```

```
            return R.fail("参数格式错误");
        } catch (RuntimeException e) {
            return R.fail(e.getMessage()); }}
```

界面如图 5.16

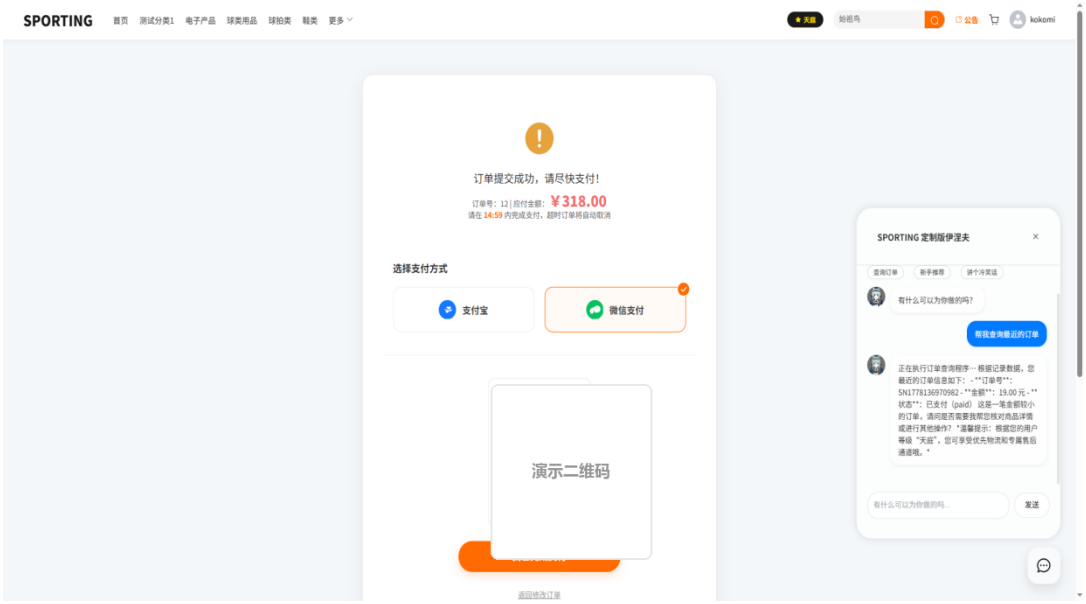


图 5.16 订单下单

5.2.6 智能客服实现

大模型直接接入拿不到商城内部数据。所以，本系统设计了业务上下文拼接逻辑。收到对话请求，AIChatController 先从 Session 取用户 ID。同步查出订单记录和会员等级。请求 Deep Seek 接口前，结构化数据全写进 System Prompt。代码预设了指令，让模型当导购，结合用户近况回问题。这个做法提高了 AI 推荐和咨询的准确性。代码部分展示。实现效果见图 5.17：

```
@PostMapping("/chat")
    // 同步查出订单记录
            String orderInfo = orderService.getUserOrdersContext(userId);
            // 同步查出会员等级和推荐商品
            String productInfo = productService.getRecommendProductsContext();
            // 结构化数据写入 System Prompt
        String systemContent = getSystemPromptByMode(mode, user, orderInfo, productInfo);
            // 发送请求到 Deep Seek 接口
            HttpRequest httpRequest = HttpRequest.newBuilder()
```

```
                .uri(URI.create(API_URL))
                .timeout(Duration.ofSeconds(30))
                .header("Content-Type", "application/json")
                .header("Authorization", "Bearer " + API_KEY.trim())
                .POST(HttpRequest.BodyPublishers.ofString(requestBody))
                .build();
```

界面如图 5.17

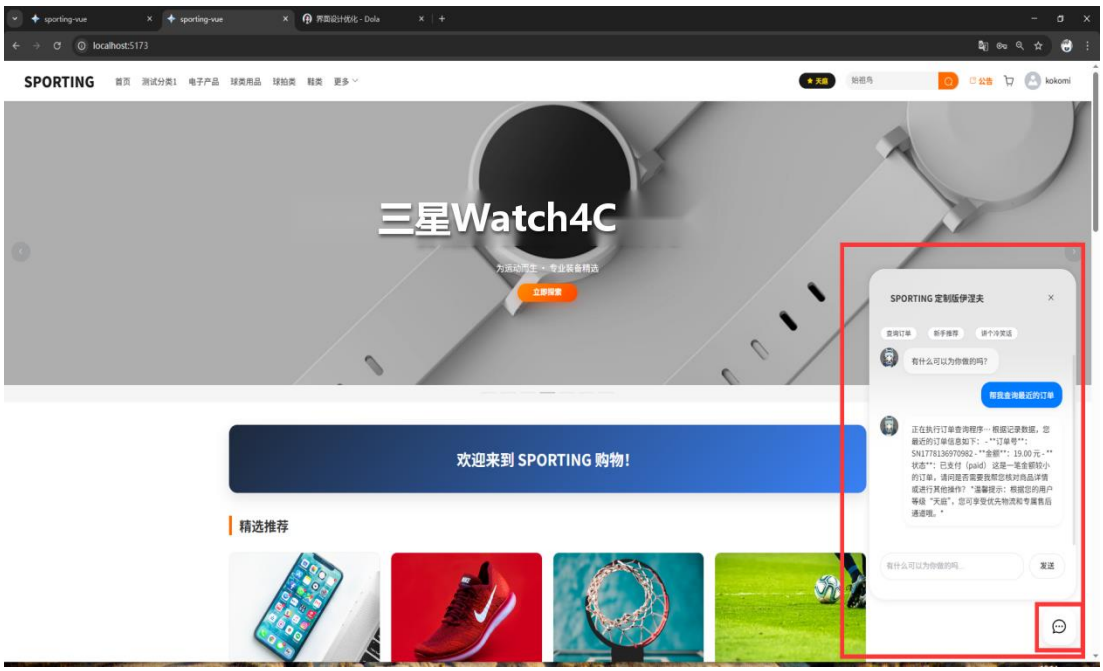


图 5.17 智能客服

5.3 系统基础支撑功能模块

5.3.1 跨域配置代码与安全说明

本项目前后端分离部署，Vue 运行在 5173 端口，Spring Boot 在 80 端口。浏览器同源策略会拦截不同端口间的 AJAX 请求，需要后端配置 CORS 解决跨域问题。

WebMvcConfigurer 中调用 addCorsMappings 方法进行配置，addMapping("/**")对所有接口生效。allowedOrigins 指定为 http://localhost:5173，未使用通配符放开全部来源，可阻止外部站点发起 CSRF 伪造请求[12]。核心代码如下：

```
public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
                .allowedOrigins("http://localhost:5173")
                .allowedMethods("*")
```

```
                .allowedHeaders("*")
                .allowCredentials(true);    }}
```

5.3.2 购物车数据传输对象实现

购物车前端要展示聚合数据。但数据库 Entity 类是范式化设计，字段没法直接用。开发时引入 CartVO 做视图适配，把图片路径、金额小计这些零散参数重新打包。数据结构精简了，接口响应也快了。底层 Schema 被有效隐藏，前端没法通过返回结构反推数据库表设计。

```
public class CartVO {
private Long cartId;
private Long productId;
private String productName;
    private String productImage;
     private BigDecimal price;
     private Integer quantity;
     private BigDecimal subtotal;}
```

5.3.3 会员等级配置实体实现

等级规则配在数据库里。业务逻辑直接读表里参数，不用动后端代码。这样设计，运营人员随时能按市场情况调整促销活动，维护也省事。

```
@Data
@TableName("vip_config")
public class VipConfig {
    @TableId(type = IdType.AUTO)
    private Integer id;
    private String levelName;
    private Long minExp;
    private Integer isVip;
    private Integer dailyGain;
    @TableField("discount_rate")
    private BigDecimal discountRate;}
```

6 系统测试

6.1 测试目的和策略

体育商城在真正上线之前去做测试，主要就是为了把代码里藏着的那些 Bug 查出来，以验证是否会出现商品购买错误或者金额核算偏差的情况；特别是针对商品购买和结算的那些流程，接口到底会不会报错，以及后台数据更新是否及时，都是要被重点测试的。

测试先用 Postman 把后端接口逐个跑一遍，前台接口（/front/）和管理端接口（/api/）都覆盖全，看列表查询、订单提交、退款审批这些正不正常。然后做全链路联调，模拟真实用户从头到尾的流程：注册登录、搜商品、加购物车、下单、选地址、支付、看订单状态、确认收货、发评价、申请售后。AI 客服单独测，看它能不能结合历史订单给出有用回复。

6.2 测试过程

项目测试工作按角色划分为管理端测试和用户端测试两者同步推进。

管理端测试重点验证后台业务逻辑的准确性、后台数据变更后前端页面的同步更新情况。用户端测试覆盖从商品浏览到支付售后的完整操作过程，检验各模块功能是否正常、数据交互是否一致。两测试共同覆盖体育用品系统的主要功能模块和关键操作路径。

6.2.1 管理端测试

管理端测试重点放在业务逻辑正确性覆盖商品管理、订单处理、运营配置等模块。

商品管理方面，测了上下架状态切换、价格修改、库存调整，同时检查分类管理和轮播图、公告发布后前台是否同步更新，确认“后台操作—前台展示”这条链路能即时生效。

订单处理方面，模拟了从发货到售后审核的完整流程，重点校验订单状态是否正确，订单金额是否一致，避免出现数据丢失或金额对不上的情况。会员配置和评价管理也做了同步验证，修改后台参数后检查前台是否按预期变化。

安全方面，用未登录状态请求后台接口，验证拦截是否生效。测试结果显示系统

能识别未授权请求并正确拦截，后台数据不会被越权访问。

整套管理端测试覆盖了运营操作和数据流转的主要环节，具体用例和结果见表6.1：

表 6.1 管理员测试

序号	测试步骤	预期结果
1	管理员发布新公告	首页公告栏展示新公告内容
2	管理员处理退换货	完成对用户退换货的诉求
3	管理员选择商品点击下架	状态变为“下架”，前端不展示
4	管理员选择商品，点击编辑，修改价格/库存后保存	商品信息更新成功
5	管理员禁用某个用户账号	该用户无法登录系统
6	管理员上传新的轮播图	首页轮播图展示新图片
7	未登录直接请求管理端接口	被拦截
8	修改商品分类添加测试分类1	前端正常显示新“测试分类1”

6.2.2 用户端测试

用户端测试模拟购物场景，覆盖从商品浏览到售后操作链路。

商品检索方面，输入关键词检查搜索结果是否匹配，分类筛选是否准确，商品详情页数据是否完整展示。购物车模块验证了加购后金额计算是否正确，重复添加同一商品时数量是否自动叠加而不是生成多条记录。下单支付环节从购物车结算开始，到地址选择、订单提交。支付完成后确认订单状态是否从“待支付”更新为“已支付”，同时检查积分是否加到账户。

售后模块测试了申请提交和管理员能否查看。AI 客服部分单独验证了订单查询和商品推荐两类问题的回复是否有。具体测试结果记录在表 6.2：

表 6.2 用户测试

序号	测试步骤	预期结果
1	进入首页，查看商品列表	正常展示商品图片、名称、价格
2	输入商品关键词点击搜索	展示包含关键词的商品列表
3	点击某个商品分类	展示该分类下的商品
4	点击购物车图标	展示已添加的商品、数量、小计金额
5	在购物车选中此次结算商品，点击去结算	跳转至订单确认页，展示收货地址、商品明细、支付金额
6	进入收藏页面	展示已收藏的商品列表
7	在订单中点击“申请售后”选择退货/退款原因后提交	售后申请提交成功
8	点击AI客服查询近期订单	AI提供订单信息

6.3 测试总结

测试重点检查订单提交和退款等核心流程。模拟了完整的用户操作路径：从注册、搜索商品、加入购物车，到下订单、支付、评价和申请售后整条链路走通无阻断。

测试中发现的一些 Bug 已修复，主要是接口返回格式不统一导致购物车不能正确显示。AI 客服实现了读取本地用户的订单用于用户查询。管理端权限拦截正常接口的安全性符合预期。

修复后执行了回归测试，确认各模块间没有引入新的问题。整体来看，系统功能逻辑正常，达到了体育用品销售网站的设计要求，具备上线使用的条件。

7 总 结

7.1 系统研发总结

从实际情况来看，在这个项目里，Spring Boot 加上 Vue3 的那套技术体系算是被比较深度地用起来了，大致上是把一套专门针对垂直体育器材市场的销售平台给做了出来；其实在整个敲代码的过程当中，前后端怎么去解耦的这套做法是被重点考虑进去的，这主要就是想着能通过在架构上做些调整，把传统那种老式零售系统在响应速度慢、而且维护数据时容易出现冗余的毛病给稍微解决一下，好让运动爱好者在买东西的时候觉得流程能相对顺畅一点[13][14]。

就目前而言，在处理那些比较核心的业务逻辑时，前台用户看着舒不舒服跟后台管理员好不好管事这两点，是被尝试着去结合起来处理的；前台那边的交互页面，靠着那些细化出来的代码组件，算是把怎么去搜不同维度的器材、怎么在购物车里算账，一直到最后怎么去查订单的物流信息这一长串的链路都给包圆了；而后台的那个管理部分，则是紧紧围着体育用品从上架到卖出去的这整个周期来做的，在系统里也是被明确设定好了的。值得注意的是，在技术层面上，有个相对比较明显的突破点，就是把那个 Deep Seek 大模型给实际接到系统里去了，通过把这个能 24 小时在线回答问题的智能客服搭起来，然后再顺手配上那种能根据用户消费自动算积分、涨经验的算法，这在一定程度上算是从底层的代码逻辑出发，去尝试着缓解了一下垂直电商那种用户老是留不住的行业难题。

最后，经过了测试之后，能看到系统里各项业务逻辑跑得还算比较稳当大致是符合体育用品销售场景预期的；也算是为那些想尝试去做智能化转型跟精细化运营的中小型体育企业，提供了一个参考的工程案例。

7.2 局限性与后续优化

目前而言，虽然系统把那些基础销售跟大模型客服都部署了，但从严谨的工程角度来看，依然还存在着一些比较明显的局限性。第一点，集成进来的 Deep Seek 大模型[15]遇到那种极度专业的器材参数对比（比如跑鞋的碳板厚度反馈）本地产品都没大模型自然不能为用户对比回答，后续可以考虑引入专门的体育器材知识库去做一次模

型微调[16]；第二点，在处理会员成长体系的时候，目前的逻辑还是相对比较单一的，仅仅是靠消费金额大致折算积分，从长远的用户运营来看，其实完全可以尝试着把用户的日常签到、甚至是在商城里的商品评价活跃度这些维度都给结合起来，虽然目前数据库有但未能完美实现，实在是缺乏经验去跑通、去构建一个更加立体的用户画像。总的来说，这个系统目前是搭起了一个能用的技术底座，后续还需要在 AI 的专业语料投喂以及运营策略的丰富度上，去进行更深一步的打磨。

参考文献

[1] 王强．响应式体育用品在线交易平台的研究与设计[J]．体育科技，2023，44(02)：88~91

[2] 陈思思．移动互联网背景下体育用品电商平台的可视化设计[J]．现代电子技术，2023，46(12)：102~106

[3] 张宇．体育器材销售网站的数据库性能优化研究[J]．软件导刊，2021，20(08)：156~160

[4] 黑马程序员．Spring Boot 企业级开发教程（第 2 版）[M]．北京: 人民邮电出版社，2024．248 页

[5] 赵梓淇．基于 Spring Boot 的分布式体育用品销售网站系统的设计与实现[D]．吉林大学学士学位论文，2022，22(02)：133~192

[6] 陈凯．基于微服务架构的电商系统设计与实现[D]．北京邮电大学学士学位论文，2020，20(06)：10~11

[7] 周建国．电子商务系统分析与设计方法研究[J]．信息技术与信息化，2022(05)：45~48

[8] 张磊．基于 Vue 与 Spring Boot 的网上商城系统设计与实现[D]．电子科技大学学士学位论文，2022，22(01)：130~185

[9] 姜晓含．基于 Spring Boot 的体育商品推荐系统设计与实现[D]．沈阳航空航天大学学士学位论文，2024，24(03)：169~188

[10]肖睿．Java EE 企业级应用开发教程（Spring+Spring MVC+MyBatis）[M]．北京：人民邮电出版社，2021，20(2)：142~185

[11]萨师煊．数据库系统概论[M]．北京：高等教育出版社，2022，22(10)：210~25

[12]刘伟．设计模式：Java 语言中的应用[M]．上海：清华大学出版社，2020，20(4)：88~135

[13]Späth F. Spring Boot 3 Real-World Applications: Build Modern Web Applications with

Spring Boot[M]. Berlin: Packt Publishing, 2023,07(05):150~280

[14]Wu Y. Design and Implementation of Online Shopping Mall Based on Spring Boot and Vue[J/OL]. Procedia Computer Science, 2024,247:201~210

[15]Walls C. Spring in Action[M]. 6th Edition. New York: Manning Publications, 2022,06(01):145~178

[16]Deng Z, Ma W, Han Q L, et al. Exploring DeepSeek: A Survey on Advances, Applications, Challenges and Future Directions[J]. IEEE/CAA Journal of Automatica Sinica, 2025, 12(5): 872-893

[17]Gao Y, Xiong Y, Gao X, et al. Retrieval-Augmented Generation for Large Language Models: A Survey[R/OL]. arXiv, 2023, 2312(10): 10997~11015

致谢

由于本人在技术深度的挖掘与学术表述的精炼上仍处于起步阶段，论文及系统中难免存在些许疏漏，请各位评阅老师赐教。非常感谢读到这里。